

PCI DSS Scope and SAQ Determination (Moyasar)

SECTION 04-compliance-ksa

TYPE memo

STATUS draft

OWNER Founder

UPDATED 2026-08-18

Purpose. Establish, on the record, what part of Fly GACA's card-payment flow falls inside PCI DSS scope, which Self-Assessment Questionnaire applies, and which obligations are live today. Written so it can be quoted directly into a B2B security questionnaire or the investor due-diligence pack.

Standing. PCI DSS is a card-scheme contractual standard, not Saudi law. It sits alongside — and does not displace — PDPL and ZATCA, which remain the governing regimes for this company. Nothing in this memo alters the position recorded in `pdpl-compliance-program-and-dpia.docx`.

1. How payment actually works

Verified against the shipping code (`FlyGACA-app/src/pages/checkout/Checkout.tsx`), not from vendor marketing:

1. `/checkout` is a page **served by Fly GACA** from its own origin.
2. That page **dynamically injects** the Moyasar Payment Form widget — `https://cdn.moyasar.com/mpf/1.16.0/moyasar.js` plus its stylesheet — via `document.createElement('script') ... document.head.appendChild(script)`.
3. The widget mounts into a Fly GACA DOM node (`#moyasar-checkout-form`).
4. The cardholder types their card, mada or Apple Pay details **into the widget**; those details post **browser → Moyasar** (`api.moyasar.com`). Apple Pay merchant validation goes to `https://api.moyasar.com/v1/applepay/initiate`.
5. Moyasar redirects back with `?id=<paymentId>`; the server confirms the payment server-to-server.

No cardholder data is transmitted to, processed by, or stored on Fly GACA servers, Firestore, or any Fly GACA log. iOS purchases go through Apple IAP / RevenueCat and are outside PCI scope entirely.

2. Why the SAQ type is not automatic

It is tempting to record "SAQ A" and move on. That would be careless, because **SAQ A eligibility turns on the integration shape, not on whether we store card data.**

- SAQ A, in its simplest reading, is for merchants who fully outsource the payment page — a **full redirect** to the processor, or an **iframe** whose contents the merchant cannot influence.

- Our integration is **neither**. It is a **JavaScript embed on a page we serve and control**. We control the DOM the widget mounts into, the script tag that loads it, and every other script on that page.

That distinction is the whole question. A merchant serving the payment page is responsible for the integrity of the scripts on it, because a compromised first-party script can read the cardholder's keystrokes regardless of where the card data is ultimately posted. This is the Magecart class of attack.

[Owner to confirm] The SAQ type must be confirmed with **Moyasar and the acquiring bank** — they determine our validation path, and the answer may be SAQ A with the script-integrity criteria, or SAQ A-EP. **This memo does not assert an answer.** What follows is true either way.

3. Obligations that are live regardless of SAQ type

PCI DSS v4.0.1 made a set of requirements mandatory from **31 March 2025**. Two bite directly on an embedded-script payment page:

Requirement	What it asks	Our position
6.4.3	Maintain an inventory of every script on the payment page; authorise each one; assure its integrity	Gap. No script inventory exists. The Moyasar script is injected at runtime with no Subresource Integrity (integrity) attribute — verified: <code>Checkout.tsx</code> sets <code>script.src</code> and <code>script.async</code> only
11.6.1	Detect unauthorised modification of the payment page's HTTP headers and content, and alert on it	Gap. No tamper-detection mechanism is deployed

These are genuine, currently-undocumented obligations. They are the substance of this memo.

4. A control discrepancy found while writing this

Fly GACA defines its Content-Security-Policy in **four** places, and they do not agree about the payment widget:

Source	Permits <code>cdn.moyasar.com</code> ?
<code>firebase.json</code> (Firebase Hosting — the canonical origin)	Yes
<code>netlify.toml</code>	Yes
<code>vercel.json</code>	Yes
<code>public/_headers</code>	No — not referenced at all

Firestore Hosting serves `flygaca.com`, so checkout works today. But `public/_headers` describes a materially **different security posture** from the origin, and any traffic served through that path would have its payment widget blocked outright by CSP. Whichever way it is resolved, a payment page whose script policy differs by edge is exactly what 6.4.3 exists to prevent.

Action: reconcile the four sources, and treat the CSP `script-src` on `/checkout` as the authoritative script allow-list — it is the cheapest available implementation of 6.4.3.

5. Scope statement — quotable

Fly GACA does not store, process, or transmit cardholder data. Card, mada, and Apple Pay credentials are captured by a Moyasar-hosted payment widget and transmitted directly from the cardholder's browser to Moyasar, a PCI DSS compliant payment service provider. Fly GACA's systems receive only a payment identifier and status. Fly GACA's residual responsibility is the integrity of the checkout page that hosts the widget and the scripts loaded onto it.

6. Actions

#	Action	Owner
1	Confirm SAQ type with Moyasar + acquirer, and record the answer here	Founder
2	Create a payment-page script inventory; authorise each entry (6.4.3)	Founder
3	Decide SRI vs. CSP-nonce for the Moyasar script, and implement (6.4.3)	Founder
4	Reconcile the four CSP sources; make <code>/checkout</code> <code>script-src</code> authoritative	Founder
5	Choose a tamper-detection approach for the payment page (11.6.1)	Founder
6	Obtain and file Moyasar's current PCI DSS Attestation of Compliance	Founder

7. Explicitly out of scope

So that silence is never later read as a gap: this memo does **not** address the remaining PCI DSS requirements — network segmentation, ASV scanning, penetration testing, or the cardholder-data lifecycle controls. Those apply to a cardholder data environment, and Fly GACA does not operate one. If the integration ever changes such that card data reaches Fly GACA infrastructure, this memo is void and the full standard applies.

Related: `pdpl-compliance-program-and-dpia.docx`, `information-security-policy.docx`, `zatca-fatoora-e-invoicing-compliance-pack.docx`, `cyber-risk-assessment-2026-08.md`.